

Proyecto de colaboración con IA — Dirigir a un agente para hacer deep learning

Peso: 40 % de la calificación final. **Formato:** individual. Puedes discutir con compañeros, pero la conversación con el agente y los entregables son tuyos. **Entregas:** dos momentos.

- **Hito 1 — especificación congelada (semana 7, la semana siguiente al Examen 1).** Entregas la especificación previa (entregable 1) antes de abrir el chat con el agente. Recibes retroalimentación breve; la especificación queda congelada y se califica junto con el resto.
- **Entrega final (semana 12).** Los cinco entregables completos.

Por qué este proyecto

En la práctica ya casi nadie escribe un loop de entrenamiento desde cero: se lo pides a un agente de IA. Pero el agente hace *exactamente lo que le pides* — si no sabes pedir una división train/val/test, pesos por clase o métricas por subgrupo, no las vas a obtener; y si no sabes leer una curva de pérdida, no vas a notar cuando el resultado esté mal. Este proyecto evalúa la siguiente habilidad: **especificar bien, verificar con escepticismo e interpretar con criterio**. El agente pone el código; tú pones el entendimiento.

Qué vas a hacer

Elige **una** de las siguientes opciones y dirígela de principio a fin usando un agente de IA (Claude, ChatGPT, Gemini, Claude Code, etc. — el que quieras). Las opciones 1–3 parten de una tarea de práctica, pero no consisten en repetirla: la tarea es la calibración (debe salirte lo que ya conoces) y el proyecto es lo que construyes encima. Las cuatro opciones tienen una carga comparable.

1. **Regularización y generalización (base: Tarea 5).** Punto de partida: el grid optimizador $\times$ weight decay en MNIST de la tarea, reproducido como calibración. Encima:

- (a) un dataset que no vimos en clase (Fashion-MNIST, KMNIST, o un dataset tabular de política pública acordado conmigo): ¿se mantienen las conclusiones del grid?;
- (b) régimen de pocos datos: $N \in \{500, 2000, 10000\}$ ejemplos, con y sin aumento de datos, y la curva de accuracy de validación contra N ;
- (c) una intervención adicional a tu elección (early stopping, dropout, label smoothing o ensamble de semillas), con su predicción escrita antes de correrla;
- (d) calibración: ECE y diagrama de confiabilidad de las dos mejores configuraciones.

Entregas curvas train/val, una tabla con val_acc, $\|\theta\|_2$ y ECE por configuración, y una recomendación: qué receta usarías con 2,000 ejemplos etiquetados y por qué.

2. **CNNs y transferencia (base: Tarea 6).** Punto de partida: el grid 3×3 de dropout $\times$ weight decay en small_data. Encima:

- (a) transfer learning con una CNN preentrenada (ResNet-18 o similar) bajo tres estrategias, features congeladas + capa lineal, fine-tuning parcial y fine-tuning completo, en el mismo régimen de pocos datos y contra tu CNN entrenada desde cero;
- (b) aumento de datos: la cuadrícula visual que verifica que las transformaciones preservan la etiqueta, más una transformación que la **rompe** a propósito (por ejemplo, flips verticales en dígitos) y su efecto medido;

- (c) análisis de errores por clase y, si el dataset lo permite, por alguna característica de la imagen (brillo, resolución, fuente);
- (d) visualización de los filtros o mapas de activación de la primera capa y qué aprendieron.

Recomendación final: con 2,000 imágenes etiquetadas y una GPU modesta, ¿desde cero o transfer learning, y con qué estrategia? ¿Con qué evidencia lo defiendes?

3. **Secuencias y modelos de lenguaje (base: Tarea 7).** Punto de partida: RNN vs. LSTM vs. GRU sobre el corpus del Quijote. Encima:

- (a) un corpus nuevo de texto público (Diario Oficial, iniciativas de ley, transcripciones de sesiones, comunicados de una dependencia), con limpieza y tokenización documentadas;
- (b) dos tokenizaciones (palabras vs. subpalabras/BPE) y la comparación honesta: por qué la perplejidad no es comparable entre ellas y con qué unidad común las comparas (bits por carácter o log-verosimilitud total del mismo texto);
- (c) split contiguo o temporal vs. split aleatorio de ventanas: mide cuánto infla la fuga la métrica de validación;
- (d) longitud de contexto $\tau \in \{5, 20, 50\}$ con BPTT truncado: costo por época contra perplejidad;
- (e) generación con temperatura y top- k , evaluada cualitativamente con una rúbrica que tú escribes.

Recomendación de despliegue: qué modelo, con qué costo por época y qué contexto, para un caso concreto donde importa la latencia (por ejemplo, autocompletado en un sistema de atención ciudadana).

Dónde conseguir el corpus (puntos de partida; revisa los términos de uso y documenta cómo lo descargaste y limpiaste):

- **Diario Oficial de la Federación:** dof.gob.mx — decretos, acuerdos y normas; texto legal formal.
- **Gaceta Parlamentaria** de la Cámara de Diputados: gaceta.diputados.gob.mx — iniciativas, dictámenes y puntos de acuerdo.
- **Diario de los Debates** (Cámara de Diputados y Senado): cronica.diputados.gob.mx, senado.gob.mx — transcripciones de sesiones; lenguaje oral y político.
- **Sistema de Información Legislativa** (SEGOB): sil.gobernacion.gob.mx — iniciativas y su estatus, útil si quieres etiquetas.
- **Buscador Jurídico de la SCJN:** bj.scjn.gob.mx — sentencias y tesis.
- **Versiónes estenográficas y comunicados** del gobierno federal: gob.mx (sección de prensa de cada dependencia) y presidente.gob.mx.
- **datos.gob.mx:** datos.gob.mx — portal de datos abiertos; varios conjuntos incluyen campos de texto libre (quejas, solicitudes, descripciones).
- **Plataforma Nacional de Transparencia:** plataformadetransparencia.org.mx — solicitudes de información y respuestas.
- **Hugging Face Datasets:** huggingface.co/datasets — busca corpus en español ya limpios (por ejemplo, Wikipedia en español, noticias, EUR-Lex en español para legislación de la UE como en Chalkidis et al.).
- **Wikipedia en español** (dumps): dumps.wikimedia.org — si quieres un corpus grande y neutro como control.

- **Project Gutenberg:** gutenberg.org — literatura en español; es de donde salió el Quijote de la tarea, sirve como línea base pero no cuenta como corpus nuevo.

Un corpus de 50,000–300,000 palabras es suficiente para este proyecto; más grande no es mejor si no cabe en tu cómputo.

4. **Opción aplicada (recomendada, un poco más ambiciosa).** Un problema de política pública con datos reales y abiertos: clasificación de texto legislativo/gubernamental, predicción de demanda de un servicio público, clasificación de imágenes satelitales con transfer learning, etc. Acuérdalo conmigo antes del Hito 1. Debe incluir un protocolo de validación y métricas por subgrupo cuando aplique.

En las opciones 1–3, cada inciso lleva su **predicción escrita** en la especificación del Hito 1, antes de correrlo; el reporte de interpretación contrasta cada predicción con lo que salió. Si algún inciso resulta imposible con tus recursos, lo documentas y lo sustituyes por otro acordado conmigo; no lo omites en silencio.

En cualquier opción el estándar experimental es el del curso: semilla fija, mismo split entre configuraciones, un factor variado a la vez, curvas + tabla resumen + interpretación escrita.

Entregables (5)

1. **Especificación previa** (1 página, escrita antes de abrir el chat). Qué vas a pedir y por qué: dataset y splits, arquitectura, configuraciones a comparar, hiperparámetros fijos, métricas y qué esperas observar teóricamente. Esta especificación se congela: entregas la versión original, con correcciones posteriores marcadas como tales.
2. **Transcripción completa** de la(s) conversación(es) con el agente, sin editar. Exporta el chat o copia todo; si usaste varias sesiones, inclúyelas todas.
3. **Reporte de verificación** (1–2 páginas). La parte más importante: ¿qué revisaste del trabajo del agente y cómo? Como mínimo: (a) verifica que el protocolo pedido se cumplió (semillas, splits, factor único); (b) revisa el código en los puntos críticos (¿la pérdida es la correcta? ¿`model.eval()` y `no_grad` en evaluación? ¿se normalizó con estadísticas solo de train?); (c) contrasta al menos un resultado contra tu predicción teórica. Documenta al menos dos errores, decisiones cuestionables o cosas que tuviste que corregir del agente — en nuestra experiencia siempre las hay; si de verdad no encontraste ninguna, explica qué revisaste para descartarlas.
4. **Interpretación de resultados** (1–2 páginas). Las preguntas de “explica por qué” de siempre: qué configuración ganó y por qué tiene sentido (o no) a la luz de la teoría del curso, limitaciones, y qué recomendarías a alguien que fuera a usar esto.
5. **Reflexión breve** (media página). ¿Qué tuviste que saber tú para que esto saliera bien? ¿Dónde el agente fue mejor que tú y dónde tú fuiste indispensable?

Rúbrica (100 pts)

Componente	Pts	Qué se evalúa
Especificación	30	Completa y teóricamente fundamentada <i>antes</i> de empezar: splits, semillas, control de factores, métricas correctas para el problema. Una especificación a la que el agente no le pueda meter un gol. Entregada en el Hito 1 (10 de los 30 pts dependen de entregarla a tiempo y congelada).
Verificación	30	Escepticismo con evidencia: revisiones concretas al código y al protocolo, errores del agente detectados y documentados, contraste resultado-vs-predicción.
Interpretación	25	Conexión con la teoría del curso, honestidad sobre limitaciones, recomendación defendible.
Reflexión y forma	15	Reflexión genuina; entrega completa (los 5 entregables), a tiempo, con la transcripción íntegra.

Lo que NO se califica: la elegancia del código (lo escribió el agente), ni qué tan buenos salieron los números. Un experimento con resultados mediocres, bien especificado, bien verificado y bien interpretado, obtiene mejor nota que uno con números espectaculares que no puedes explicar.

Descalificación automática: entregar sin la transcripción completa, o una transcripción que no corresponde a los resultados reportados.

Consejos

- Pídele al agente el plan antes que el código y críticalo contra tu especificación.
- Pide instrumentación desde el inicio (curvas train/val, normas de gradiente, tablas) — es más barato que reconstruirla después.
- Cuando algo se vea demasiado bien (val acc > train acc, pérdida en cero), sospecha primero de fuga de datos o de un bug de evaluación.
- Los notebooks del curso (`notebooks/`) son tu referencia de qué debería aparecer en el código; úsalos para auditar lo que el agente produzca.